

VICTORIA PARK ACADEMY SHENZHEN

CIE IGCSE COMPUTER SCIENCE (0478) — YEAR 9 SEMESTER 2

ASSESSMENT MATERIALS & REPORTING PACK

Document Reference: VPA-CS-Y9S2-ASSESS-v1.0

Academic Year: 2025–2026

Teaching Weeks: 18 weeks

Assessment Count : 4 in-class quizzes + 1 end-of-semester mock exam

This document contains ALL assessment materials for Year 9 Semester 2:

- Quiz 1 — Algorithms & Pseudocode (Week 3, 15 min, 20 marks)
- Quiz 2 — Python Programming (Week 7, 20 min, 25 marks)
- Quiz 3 — Data Structures (Arrays & Strings) (Week 10, 15 min, 20 marks)
- Quiz 4 — File Handling & Functions (Week 14, 15 min, 20 marks)
- Mock Exam — End-of-Semester Paper 2 Style (Week 18, 60 min, 50 marks)
- Student Report Template
- Bilingual Glossary Poster (50+ terms by unit)

ALL materials include complete questions, mark allocations, and answer keys.

Quizzes should be printed and stored securely. Answer keys are TEACHER ONLY.

QUIZ 1 — ALGORITHMS & PSEUDOCODE

WEEK 3 | 15 MINUTES | 20 MARKS

TEACHER NOTES:

- Print one paper per student. Do not release digitally.
- Allow use of pseudocode reference card (Appendix A of Overview).
- Bilingual word bank provided on last page of this quiz.
- Calculators NOT permitted.

STUDENT NAME: _____ CLASS: _____ DATE: _____

SECTION A — MULTIPLE CHOICE (5 marks)

Choose the correct answer. Write A, B, C, or D in the box.

1. Which flowchart symbol is used for a decision?

- ☐ A. Rectangle ☐ B. Parallelogram ☐ C. Diamond ☐ D. Oval

[1 mark]

2. What is the purpose of decomposition in problem solving?

- ☐ A. To remove all variables from a program
- ☐ B. To break a complex problem into smaller sub-problems
- ☐ C. To convert pseudocode into Python
- ☐ D. To test a program with different data

[1 mark]

1. In CIE pseudocode, which keyword is used to start a count-controlled loop?

- ☐ A. WHILE
- ☐ B. REPEAT
- ☐ C. FOR
- ☐ D. DO

[1 mark]

4. What does the arrow symbol (←) mean in pseudocode?

- ☐ A. Compare two values
- ☐ B. Output a value to the screen
- ☐ C. Assign a value to a variable
- ☐ D. Input a value from the user

[1 mark]

5. Which of the following is an example of abstraction?

- ☐ A. Writing every detail of how a car engine works
- ☐ B. Representing a person using only their name and age for a program
- ☐ C. Drawing a flowchart with every possible path shown
- ☐ D. Testing a program with every possible input value

[1 mark]

SECTION B — SHORT ANSWER (10 marks)

2. (a) Draw the flowchart symbol for each of the following operations:

(i) Start / End [1 mark]

(ii) Input / Output [1 mark]

(iii) Process / Calculation [1 mark]

(b) Name the missing flowchart symbol: A rectangle with double vertical

lines is used for a _____. [1 mark]

3. Write pseudocode for an algorithm that does the following:

- Ask the user to enter their age
- If the age is 18 or over, output "Adult"
- Otherwise, output "Minor"

Use correct CIE pseudocode syntax. [3 marks]

4. Trace through the following algorithm using a trace table.

```

x ← 5
y ← 3
WHILE x < 20 DO
    x ← x + y
    y ← y + 1
    OUTPUT x
ENDWHILE

```

[3 marks]

5. A shop offers a discount based on how much a customer spends:

- Write pseudocode to:

- Use these variables: Spend, Discount, FinalAmount

QUIZ 1 — ANSWER KEY (TEACHER ONLY)

6. C — Diamond (decision symbol) [1 mark]
7. B — Breaking a complex problem into smaller sub-problems [1 mark]
8. C — FOR [1 mark]
9. C — Assign a value to a variable [1 mark]
10. B — Representing a person using only relevant details [1 mark]

11. (a) (i) Oval / Rounded rectangle [1 mark]
(ii) Parallelogram [1 mark]
(iii) Rectangle [1 mark]
(b) Pre-defined process / Subroutine / Call to another module [1 mark]
(Accept: "subroutine", "function call", "module")

7. Mark allocation — 3 marks total:

[1 mark] for correct INPUT statement

[1 mark] for correct IF/ELSE structure with correct condition

[1 mark] for correct OUTPUT statements in both branches

Exemplar answer:

```
INPUT "Enter your age: ", Age      ← [1]
```

```
IF Age >= 18 THEN                  ← [1]
```

```
    OUTPUT "Adult"                 ← [1]
```

```
ELSE
```

```
    OUTPUT "Minor"
```

```
ENDIF
```

(Accept equivalent conditions: Age > 17)

12. Trace table — 3 marks total (1 mark per correct row after initial):

x	y	OUTPUT	
5	3	—	← Initial values (given)
8	4	8	← [1]
12	5	12	← [1]
17	6	17	← [1]
23	7	23	← (loop ends as 23 $\nless$ 20)

Award marks for correct rows. Deduct [1] if column headers incorrect.

Accept trace tables with loop condition column included.

SECTION C — PROBLEM SOLVING (5 marks)

9. Mark allocation:

[1 mark] Input amount spent

[2 marks] Correct nested IF with all three conditions

[2 marks] Correct calculations and output

Exemplar answer:

```
INPUT "Enter amount spent: ", Spend      ← [1]
```

```
IF Spend >= 100 THEN                    ← [1]
```

```
    Discount ← Spend * 0.20             ← [1]
```

```
ELSE IF Spend >= 50 THEN
```

```
    Discount ← Spend * 0.10
```

```
ELSE
```

```
Discount ← 0
ENDIF
FinalAmount ← Spend - Discount      ← [1]
OUTPUT "Final amount: ", FinalAmount ← [1]
```

Alternative approaches accepted:

- Using multiplication: FinalAmount ← Spend * 0.80 (for 20% off)
 - Using separate IF statements (not nested)
 - Any logically correct structure that produces correct output
- [-1 mark] for each missing ENDIF (pseudocode syntax error)

QUIZ 1 TOTAL: 20 marks

QUIZ 2 — PYTHON PROGRAMMING (SELECTION & ITERATION)

WEEK 7 | 20 MINUTES | 25 MARKS

TEACHER NOTES:

- Print one paper per student. Thonny IDE should NOT be open during quiz.
- Students write Python code on paper — syntax errors penalised lightly.
- Bilingual word bank provided on last page.
- Calculators NOT permitted.

STUDENT NAME: _____ CLASS: _____ DATE: _____

SECTION A — MULTIPLE CHOICE (5 marks)

Choose the correct answer. Write A, B, C, or D in the box.

1. What data type is the value 3.14 in Python?

- ☐ A. int ☐ B. float ☐ C. str ☐ D. bool

[1 mark]

2. What is the output of the following code?

```
x = 7
if x > 5:
    print("A")
else:
    print("B")
```

- ☐ A. A ☐ B. B ☐ C. AB ☐ D. Nothing

[1 mark]

3. What is the output of the following loop?

```
for i in range(3):
```

```
    print(i)
```

- ☐ A. 1 2 3 ☐ B. 0 1 2 ☐ C. 0 1 2 3 ☐ D. 3 2 1

[1 mark]

13. Which operator gives the remainder of a division in Python?

- ☐ A. / ☐ B. // ☐ C. % ☐ D. **

[1 mark]

5. What will the following code output?

```
count = 0
```

```
while count < 3:
```

```
    count = count + 1
```

```
    print(count)
```

- ☐ A. 0 ☐ B. 2 ☐ C. 3 ☐ D. 4

[1 mark]

SECTION B — CODE READING (10 marks)

14. (a) What is the output of the following code?

```
name = "Computer"
```

```
print(len(name))
```

```
print(name[0])
```

```
print(name[-1])
```

Write the THREE lines of output:

[3 marks]

(b) The following code contains THREE errors. Find and correct them.

```
age = input("Enter your age: ")
```

```
if age > 13
```

```
    print("You are a teenager")
```

```
else
```

```
    print("You are not a teenager")
```

Error 1: _____ [1 mark]

Error 2: _____ [1 mark]

Error 3: _____ [1 mark]

7. Fill in the missing parts of this program.

The program should keep asking the user to guess a number until they guess correctly (the number is 7).

```
secret = 7
guess = _____ # Line A [1 mark]
while _____: # Line B [1 mark]
    guess = int(input("Guess again: "))
print("_____") # Line C [1 mark]
```

SECTION C — CODE WRITING (10 marks)

8. Write a Python program that:

- Asks the user to enter a number
- If the number is positive, prints "Positive"
- If the number is negative, prints "Negative"
- If the number is zero, prints "Zero"

Include appropriate comments. [4 marks]

15. Write a Python program that uses a FOR loop to print the 7 times table from 1 to 12, in this format:

```
1 x 7 = 7
2 x 7 = 14
...
12 x 7 = 84 [6 marks]
```

END OF QUIZ 2

QUIZ 2 — ANSWER KEY (TEACHER ONLY)

SECTION A — MULTIPLE CHOICE (5 marks)

16. B — float [1 mark]
17. A — A (7 > 5 is True, so "A" is printed) [1 mark]
18. B — 0 1 2 (range(3) generates 0, 1, 2) [1 mark]
19. C — % (modulo operator) [1 mark]
20. C — 3 (loop runs while count < 3: count becomes 1, 2, then 3) [1 mark]

SECTION B — CODE READING (10 marks)

21. (a) Output — 3 marks (1 mark per correct line):

```
8 [1 mark]
C [1 mark]
r [1 mark]
```

(Accept: 8, C, r on separate lines)

(b) Three errors — 3 marks (1 mark each):

Error 1: Missing colon (:) after if age > 13 [1 mark]

Correct: if age > 13:

Error 2: Missing colon (:) after else [1 mark]

Correct: else:

Error 3: age is a string from input(), needs int() conversion [1 mark]

Correct: age = int(input("Enter your age: "))

(Accept: float() conversion as well)

Award [1 mark] for identifying each error even if correction is slightly imprecise. Award mark for "indentation missing" only if student also mentions the colon.

7. Fill in the blanks — 3 marks:

Line A: int(input("Enter your guess: ")) or 0 [1 mark]

(Accept: 0, any integer, or input converted to int)

Must initialise guess to a value that enters the loop.

Line B: guess != secret [1 mark]

(Accept: guess != 7, guess is not secret, not(guess == secret))

(Accept equivalent: while guess < secret or guess > secret:)

Line C: "Correct!" or "Well done!" or "You guessed it!" [1 mark]

(Any appropriate congratulatory message accepted)

Exemplar complete code:

```
secret = 7
guess = int(input("Enter your guess: "))
while guess != secret:
    guess = int(input("Guess again: "))
print("Correct!")
```

SECTION C — CODE WRITING (10 marks)

8. Number checker program — 4 marks:

[1 mark] Correct input with type conversion (int() or float())

[1 mark] IF/ELIF/ELSE structure with correct conditions

[1 mark] All three print statements correct

[1 mark] Code is syntactically correct Python (minor syntax errors: -½)

Exemplar answer:

```
# Get number from user [1]
num = float(input("Enter a number: "))
```



```

if num > 0:                                [1]
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")                          [1]

```

Deduct [½ mark] for:

- Missing type conversion on input
- Using separate IF statements instead of IF/ELIF/ELSE
- Missing colon after condition

22.7 times table — 6 marks:

[1 mark] Correct FOR loop structure (for ... in range(...):)

[1 mark] range() generates 1 to 12 inclusive (range(1, 13))

[1 mark] Loop variable used correctly (e.g., i or num)

[1 mark] Correct multiplication calculation (i * 7)

[1 mark] Output in correct format: "1 x 7 = 7" (or equivalent)

[1 mark] Code is syntactically correct Python

Exemplar answer:

```

for i in range(1, 13):                    [1]
    answer = i * 7                        [1]
    print(i, "x 7 =", answer)            [1]

```

Alternative:

```

for i in range(1, 13):
    print(str(i) + " x 7 = " + str(i * 7))

```

Deduct [½ mark] for:

- range(12) instead of range(1, 13) (would print 0–11)
- range(1, 12) instead of range(1, 13) (would miss 12)
- Wrong format string (but still produces readable output: –½)

QUIZ 2 TOTAL: 25 marks

QUIZ 3 — DATA STRUCTURES (ARRAYS & STRINGS)

WEEK 10 | 15 MINUTES | 20 MARKS

TEACHER NOTES:

- Print one paper per student. Thonny IDE should NOT be open during quiz.
- Students should know Python list syntax and common string methods.
- Remind students: Python lists use 0-based indexing.
- Bilingual word bank provided on last page.

STUDENT NAME: _____ CLASS: _____ DATE: _____

SECTION A — MULTIPLE CHOICE (5 marks)

Choose the correct answer. Write A, B, C, or D in the box.

23. In Python, what is the index of the first element in a list?

- ☐ A. 1 ☐ B. 0 ☐ C. -1 ☐ D. None

[1 mark]

2. What is the output of the following code?

```
fruits = ["apple", "banana", "cherry"]  
print(fruits[1])
```

- ☐ A. apple ☐ B. banana ☐ C. cherry ☐ D. Error

[1 mark]

24. Which string method returns a copy of the string in uppercase letters?

- ☐ A. upper() ☐ B. uc() ☐ C. capitalize() ☐ D. big()

[1 mark]

4. What is the output of the following code?

```
word = "Hello"  
print(word[1:4])
```

- ☐ A. Hello ☐ B. ell ☐ C. ello ☐ D. Hel

[1 mark]

5. What does the following code do?

```
scores = [85, 90, 78]  
scores.append(92)
```

- ☐ A. Removes 92 from the list
☐ B. Adds 92 to the end of the list
☐ C. Adds 92 to the start of the list
☐ D. Replaces the last element with 92

[1 mark]

SECTION B — CODE READING (7 marks)

25. Trace the following code and write the final state of the list.

```
nums = [4, 2, 8, 1]  
nums[0] = nums[0] + nums[2]  
nums.append(5)  
nums.remove(2)
```

```
nums.sort()
```

Final list: _____ [3 marks]

26. (a) What is the output of the following code?

```
text = "Computer Science"
```

```
print(len(text))
```

```
print(text[0:8])
```

```
print(text[-7:])
```

Line 1: _____

Line 2: _____

Line 3: _____ [3 marks]

(b) What is the ASCII code of the character 'A'? [1 mark]

Answer: _____

SECTION C — CODE WRITING (8 marks)

8. Write a Python program that:

- Creates a list of 5 colours: ["red", "blue", "green", "yellow", "purple"]
- Asks the user to enter a colour name
- Uses a FOR loop to check if the colour is in the list
- Prints "Found!" if it is in the list, or "Not found!" if it is not

Include comments. [5 marks]

9. Write a Python program that:

- Asks the user to enter their first name and last name (separately)
- Joins them with a space in between
- Converts the full name to uppercase
- Prints the full name and its length

Include comments. [3 marks]

END OF QUIZ 3

QUIZ 3 — ANSWER KEY (TEACHER ONLY)

SECTION A — MULTIPLE CHOICE (5 marks)

27.B — 0 (Python uses zero-based indexing) [1 mark]

28.B — banana (index 1 is the second element) [1 mark]

29.A — upper() [1 mark]

30.B — ell (indices 1, 2, 3 — characters 'e', 'l', 'l') [1 mark]

31.B — Adds 92 to the end of the list [1 mark]

SECTION B — CODE READING (7 marks)

6. Trace table analysis — 3 marks total:

Initial: nums = [4, 2, 8, 1]

After nums[0] = nums[0] + nums[2]: nums = [12, 2, 8, 1] (4+8=12)

After nums.append(5): nums = [12, 2, 8, 1, 5]

After nums.remove(2): nums = [12, 8, 1, 5] (removes first 2)

After nums.sort(): nums = [1, 5, 8, 12]

Final list: [1, 5, 8, 12] [3 marks]

Award marks:

[1 mark] for correct list after first two operations

[1 mark] for correctly handling remove(2) and sort()

[1 mark] for completely correct final answer

Deduct [½ mark] if answer shows [1, 8, 5, 12] or similar near-miss

32. (a) Output — 3 marks (1 per correct line):

Line 1: 16 [1 mark]

Line 2: Computer [1 mark]

Line 3: Science [1 mark]

(Note: len("Computer Science") = 16, including the space)

(b) 65 [1 mark]

(Accept: 065)

SECTION C — CODE WRITING (8 marks)

8. Colour search program — 5 marks:

[1 mark] Correct list initialisation with 5 colours

[1 mark] Input statement to get colour from user

[1 mark] FOR loop that iterates through the list

[1 mark] Correct IF comparison inside the loop

[1 mark] Correct output messages and loop structure works

Exemplar answer:

```
# List of colours
```

```
colours = ["red", "blue", "green", "yellow", "purple"] [1]
```

```
# Get user input
```

```
search = input("Enter a colour: ") [1]
```

```
found = False
```

```
for colour in colours: [1]
```

```
    if colour == search: [1]
```

```
        found = True
if found:
    print("Found!")
else:
    print("Not found!")
```

Alternative using 'in' operator (acceptable, award full marks):

```
colours = ["red", "blue", "green", "yellow", "purple"]
search = input("Enter a colour: ")
if search in colours:
    print("Found!")
else:
    print("Not found!")
```

Deduct [½ mark] for:

- Missing comments
- Using IF without a loop (if required by question)

9. Name formatter — 3 marks:

[1 mark] Two separate input statements for first and last name

[1 mark] Concatenation with space, conversion to uppercase

[1 mark] Correct output of full name and length

Exemplar answer:

```
first = input("Enter first name: ")          [1]
last = input("Enter last name: ")
full_name = (first + " " + last).upper()     [1]
print("Full name:", full_name)
print("Length:", len(full_name))             [1]
```

Alternative for uppercase:

```
full_name = first + " " + last
print("Full name:", full_name.upper())
print("Length:", len(full_name))
```

QUIZ 3 TOTAL: 20 marks

QUIZ 4 — FILE HANDLING & FUNCTIONS

WEEK 14 | 15 MINUTES | 20 MARKS

TEACHER NOTES:

- Print one paper per student. Thonny IDE should NOT be open during quiz.

- Students write Python code on paper.
- Remind students of key file mode strings: "r", "w", "a"
- Bilingual word bank provided on last page.

STUDENT NAME: _____ CLASS: _____ DATE: _____

SECTION A — MULTIPLE CHOICE (5 marks)

Choose the correct answer. Write A, B, C, or D in the box.

33. Which file mode in Python is used to add data to the end of an existing file without deleting the current content?

- ☐ A. "r" ☐ B. "w" ☐ C. "a" ☐ D. "x"

[1 mark]

2. What does the following code do?

```
myFile = open("data.txt", "r")  
line = myFile.readline()  
myFile.close()
```

- ☐ A. Writes a line to the file
☐ B. Reads the first line from the file
☐ C. Deletes the first line from the file
☐ D. Appends a line to the file

[1 mark]

3. In Python, where can a local variable be accessed?

- ☐ A. Anywhere in the program
☐ B. Only inside the function where it is defined
☐ C. Only in the main program, not in functions
☐ D. In any function that is called after it

[1 mark]

4. What keyword is used in Python to define a function?

- ☐ A. function ☐ B. define ☐ C. func ☐ D. def

[1 mark]

5. What is the output of the following code?

```
def multiply(a, b):  
    return a * b  
  
result = multiply(4, 5)
```

```
print(result)
```

☐ A. 9 ☐ B. 20 ☐ C. 45 ☐ D. 1

[1 mark]

SECTION B — CODE READING (7 marks)

34. The following Python code is supposed to read numbers from a file and print the total. It contains THREE errors. Find and correct them.

```
file = open("numbers.txt" "r")
```

```
total = 0
```

```
for line in file
```

```
    total = total + int(line)
```

```
print("Total:", total)
```

```
close(file)
```

Error 1: _____ [1 mark]

Error 2: _____ [1 mark]

Error 3: _____ [1 mark]

7. Trace the following code. What is the final output?

```
def calculate(a, b):
```

```
    c = a + b
```

```
    return c
```

```
def main():
```

```
    x = 3
```

```
    y = 7
```

```
    result = calculate(x, y)
```

```
    print(result)
```

```
    print(calculate(10, 5))
```

```
main()
```

Line 1 output: _____ [1 mark]

Line 2 output: _____ [1 mark]

What happens if we add `print(c)` at the end of `main()`?

_____ [1 mark]

SECTION C — CODE WRITING (8 marks)

8. Write a Python function called `is_even(number)` that:

- Takes one integer parameter called `number`
- Returns `True` if the number is even, and `False` if it is odd

Include a docstring comment explaining what the function does.

Then write TWO lines of code that call your function with the value 8 and print the result. [4 marks]

9. Write a Python program that:

- Opens a file called "students.txt" in write mode
- Writes three student names (one per line):
"Alice", "Bob", "Charlie"

- Closes the file
- Then reopens the file in read mode
- Reads and prints each line

Include comments. [4 marks]

END OF QUIZ 4

QUIZ 4 — ANSWER KEY (TEACHER ONLY)

SECTION A — MULTIPLE CHOICE (5 marks)

35.C — "a" (append mode) [1 mark]

36.B — Reads the first line from the file [1 mark]

37.B — Only inside the function where it is defined [1 mark]

38.D — def [1 mark]

39.B — 20 ($4 * 5 = 20$) [1 mark]

SECTION B — CODE READING (7 marks)

6. Three errors — 3 marks (1 mark each):

Error 1: Missing comma between filename and mode in open()

Correct: file = open("numbers.txt", "r") [1 mark]

(Accept: using open() with 'with' statement)

Error 2: Missing colon (:) after the FOR loop

Correct: for line in file: [1 mark]

Error 3: close(file) should be file.close()

Correct: file.close() [1 mark]

(Accept: using 'with open(...)' which auto-closes)

Alternative acceptable answers:

- Using 'with' statement to handle file opening/closing properly
- Not converting line to int() before adding (accept as valid error)
- Missing .strip() on line before int() conversion (accept as valid error)

7. Trace analysis — 3 marks:

Line 1 output: 10 ($3 + 7 = 10$) [1 mark]

Line 2 output: 15 (10 + 5 = 15) [1 mark]

What happens if print(c) is added: [1 mark]

"Error" / "NameError" / "c is not defined" / "c is a local variable"

(Any answer that indicates c is not accessible outside calculate())

Accept: "It would cause an error because c is a local variable"

"c only exists inside the calculate function"

SECTION C — CODE WRITING (8 marks)

40. is_even function — 4 marks:

[1 mark] Correct function definition: def is_even(number):

[1 mark] Correct logic using % operator (number % 2 == 0)

[1 mark] Correct return values (True for even, False for odd)

[1 mark] Docstring comment included + correct function call

Exemplar answer:

```
def is_even(number):
```

```
    """Returns True if number is even, False otherwise.""" [1]
```

```
    if number % 2 == 0: [1]
```

```
        return True [1]
```

```
    else:
```

```
        return False
```

```
print(is_even(8)) [1]
```

Alternative (more concise):

```
def is_even(number):
```

```
    """Returns True if number is even."""
```

```
    return number % 2 == 0
```

```
result = is_even(8)
```

```
print(result)
```

Deduct [½ mark] for:

- Missing docstring
- print() inside the function instead of return
- Indentation errors

9. File write and read — 4 marks:

[1 mark] Open file in write mode and write three names (one per line)

[1 mark] File properly closed (or 'with' used)

[1 mark] File reopened in read mode and lines read/printed

[1 mark] Comments included, code is syntactically correct

Exemplar answer:

```
# Write names to file
file = open("students.txt", "w")           [1]
file.write("Alice\n")
file.write("Bob\n")
file.write("Charlie\n")
file.close()                               [1]

# Read names from file
file = open("students.txt", "r")           [1]
for line in file:
    print(line.strip())
file.close()
```

Alternative using 'with' (award full marks):

```
with open("students.txt", "w") as file:
    file.write("Alice\n")
    file.write("Bob\n")
    file.write("Charlie\n")

with open("students.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Deduct [½ mark] for:

- Missing newlines (\n) when writing
- Not closing files
- Not using strip() resulting in extra blank lines in output

QUIZ 4 TOTAL: 20 marks

END-OF-SEMESTER MOCK EXAMINATION

PAPER 2 STYLE — WEEK 18 | 60 MINUTES | 50 MARKS

TEACHER NOTES:

- This is a full Paper 2 style mock examination for Year 9 Semester 2.
- Duration: 60 minutes. Print as a formal examination paper.
- No calculators, no notes, no pseudocode reference cards.
- Students answer on this paper in spaces provided.
- Section C requires BOTH pseudocode AND Python — this is intentional stretch.

STUDENT NAME: _____ CLASS: _____ DATE: _____

SECTION A — MULTIPLE CHOICE & SHORT ANSWER (15 marks)

A1. Which of the following is a correct variable assignment in pseudocode?

- ☐ A. $5 \leftarrow x$
- ☐ B. $x = 5$
- ☐ C. $x \leftarrow 5$
- ☐ D. SET x TO 5

[1 mark]

A2. What is the output of the following pseudocode?

```
x ← 10
y ← 3
z ← x DIV y
OUTPUT z
```

- ☐ A. 3.33
- ☐ B. 3
- ☐ C. 1
- ☐ D. 30

[1 mark]

A3. In a flowchart, a rectangle represents:

- ☐ A. A decision
- ☐ B. Input or output
- ☐ C. A process or calculation
- ☐ D. The start or end

[1 mark]

A4. Which Python data type would be most suitable for storing a person's telephone number (e.g., "07555123456")?

- ☐ A. int
- ☐ B. float
- ☐ C. string
- ☐ D. bool

[1 mark]

A5. What is the final value of total after the following code runs?

```
total = 0
for i in range(1, 6):
    total = total + i
```

- ☐ A. 10
- ☐ B. 15
- ☐ C. 5
- ☐ D. 0

[1 mark]

A6. (a) State the difference between a syntax error and a logic error.

[2 marks]

(b) Give ONE example of erroneous (abnormal) test data for a program that asks the user to enter their age (0–120). [1 mark]

A7. (a) Write the Python code to create an empty list called scores. [1 mark]

(b) Write ONE line of Python code to add the value 85 to the list. [1 mark]

(c) Write a Python FOR loop that prints every item in the list

[2 marks]

(a) `print(len("Hello World"))` [1 mark]

(b) `print("Hello World"[6:])` [1 mark]

(c) `print("Hello" + "World")` [1 mark]

B9. Trace through the following pseudocode using a trace table.

```
num ← 1
total ← 0
REPEAT
    total ← total + num
    num ← num + 2
UNTIL num > 8
OUTPUT "Total:", total
OUTPUT "Final num:", num
```

num	total	OUTPUT
-----	-------	--------

Final output line 2: _____ [1 mark]

B10. Study the following Python function and answer the questions.

```
if score >= 90:
    return "A"
elif score >= 80:
    return "B"
elif score >= 70:
    return "C"
elif score >= 60:
    return "D"
else:
    return "F"
```

(b) What is returned by `check_grade(59)`? [1 mark]

(c) Explain why the order of the conditions matters in this

function. What would happen if the conditions were checked in reverse order (starting with score ≥ 60)? [3 marks]

B11. The following Python program has a logic error.

```
def find_max(numbers):  
    maximum = 0  
    for num in numbers:  
        if num > maximum:  
            maximum = num  
    return maximum
```

(a) What does the function return when called with `find_max([3, 7, 2])`? [1 mark]

(b) What does the function return when called with `find_max([-5, -2, -8])`? [1 mark]

(c) Explain why the result in (b) is incorrect, and state what

initial value should be used instead of 0. [3 marks]

SECTION C — PROBLEM SOLVING (20 marks)

C12. Password Validator

Write BOTH pseudocode AND Python code for the following problem:

A program needs to validate a password. The password rules are:

- Must be at least 8 characters long
- Must contain at least one number

The program should:

(a) Ask the user to enter a password [1 mark]

(b) Check if the password is at least 8 characters long [2 marks]

(c) Check if the password contains at least one number [3 marks]

(d) Output "Valid" if both rules pass, or "Invalid" with the reason(s) if not [2 marks]

PSEUDOCODE (7 marks):

PYTHON (7 marks):

C13. Student Marks Processor

Write BOTH pseudocode AND Python code for the following problem:

A teacher wants to process a list of student exam marks. The program should:

(a) Store these marks in a list: 65, 72, 58, 91, 84, 76, 55, 88 [1 mark]

(b) Calculate the average (mean) mark [2 marks]

(c) Count how many students passed (mark ≥ 60) [2 marks]

(d) Output:

- All the marks

- The average mark (rounded to 1 decimal place)
- How many students passed
- The highest mark [2 marks]

PSEUDOCODE (7 marks):

PYTHON (7 marks):

END OF MOCK EXAMINATION

MOCK EXAM — MARK SCHEME (TEACHER ONLY)

SECTION A — MULTIPLE CHOICE & SHORT ANSWER (15 marks)

A1. C — $x \leftarrow 5$ [1 mark]

A2. B — 3 (10 DIV 3 = 3, integer division) [1 mark]

A3. C — A process or calculation [1 mark]

A4. C — string (leading zero would be lost in int) [1 mark]

A5. B — 15 (0+1+2+3+4+5 = 15) [1 mark]

A6. (a) — 2 marks:

[1 mark] Syntax error: A mistake in the grammar/rules of the language;
the program will not run. Examples: missing colon, spelling
mistake in keyword, missing bracket.

[1 mark] Logic error: The program runs but produces incorrect results.
The code is valid but the algorithm is wrong. Examples:
wrong condition, wrong calculation, wrong variable used.

(b) — 1 mark:

Any value outside 0–120, e.g., -5, 150, "abc", 12.5, empty input
(Any clearly invalid input accepted)

A7. (a) scores = [] [1 mark]

(Accept: scores = list())

(b) scores.append(85) [1 mark]

(Accept: scores = scores + [85], scores.insert(0, 85))

(c) — 2 marks:

[1 mark] Correct FOR loop structure

[1 mark] Correct print statement inside loop

Exemplar:

for score in scores: [1]

print(score) [1]

Alternative:

for i in range(len(scores)):

print(scores[i])

- A8. (a) 11 [1 mark]
 (b) World [1 mark]
 (c) HelloWorld (or HelloWorld with no space) [1 mark]
 (Accept: "HelloWorld")

SECTION A TOTAL: 15 marks

SECTION B — CODE READING & TRACING (15 marks)

B9. Trace table — 5 marks total:

Award marks for correct trace table rows:

[1 mark] Initial row correct: num=1, total=0, OUTPUT=—

[1 mark] Row 2: num=3, total=1

[1 mark] Row 3: num=5, total=4

[1 mark] Row 4: num=7, total=9

[1 mark] Row 5: num=9, total=16

[1 mark] Correct final outputs (any two of above, plus output lines)

num	total	OUTPUT
1	0	— ← [1] Initial
3	1	— ← [1]
5	4	— ← [1]
7	9	— ← [1]
9	16	— ← [1] (loop ends: 9 > 8)

Final output line 1: Total: 16 [½ mark]

Final output line 2: Final num: 9 [½ mark]

(5 marks total: 4 for table + 1 for outputs)

B10. (a) "B" [1 mark]

(b) "F" [1 mark]

(c) — 3 marks:

[1 mark] The order matters because conditions are checked sequentially; once a condition is met, the function returns immediately.

[1 mark] If checked in reverse (score >= 60 first), a score of 85 would return "D" instead of "B" because the first matching condition is score >= 60.

[1 mark] The function would not work correctly because higher scores would be caught by lower-grade conditions first.

Exemplar explanation:

"The conditions are checked from top to bottom. A score of 85 is ≥ 60 , so if that condition came first, it would return 'D' immediately and never check if the score is also ≥ 80 or ≥ 90 . The highest threshold must be first."

B11. (a) 7 [1 mark]

(b) 0 (or incorrectly returns 0) [1 mark]

(c) — 3 marks:

[1 mark] The function returns 0 because maximum starts at 0, and all numbers in the list are negative (less than 0).

[1 mark] So no number is ever greater than maximum, and 0 is returned as the "maximum" — which is wrong.

[1 mark] The initial value should be the first element of the list: `maximum = numbers[0]` (or set maximum to a very small number, or the first list element).

Exemplar:

"The function returns 0 because all the numbers are negative, so none are greater than 0. The initial value should be set to the first element in the list, e.g., `maximum = numbers[0]`, or use negative infinity."

SECTION B TOTAL: 15 marks

SECTION C — PROBLEM SOLVING (20 marks)

C12. Password Validator (10 marks: 7 pseudocode + 3 Python)

PSEUDOCODE — 7 marks:

[1 mark] Input password from user

[1 mark] Check length ≥ 8 correctly

[1 mark] Initialise a flag/variable for number found

[1 mark] Loop through each character to check for a digit

[1 mark] Correct logic to detect at least one number

[1 mark] Correct IF structure for final validation

[1 mark] Appropriate OUTPUT statements for valid/invalid with reason

Exemplar pseudocode:

```
INPUT "Enter password: ", password          [1]
validLength ← LENGTH(password) >= 8        [1]
hasNumber ← FALSE                          [1]
FOR i ← 1 TO LENGTH(password)              [1]
    ch ← SUBSTRING(password, i, 1)
```



```

IF ch >= "0" AND ch <= "9" THEN
    hasNumber ← TRUE          [1]
ENDIF
NEXT i
IF validLength AND hasNumber THEN          [1]
    OUTPUT "Valid"
ELSE
    OUTPUT "Invalid"
    IF NOT validLength THEN OUTPUT "Too short" ENDIF
    IF NOT hasNumber THEN OUTPUT "Needs a number" ENDIF
ENDIF          [1]

```

PYTHON — 3 marks:

(Note: Python section is only 3 marks — pseudocode is the primary focus)

[1 mark] Input and length check

[1 mark] Loop/check for number in password

[1 mark] Correct output with reasons

Exemplar Python:

```

password = input("Enter password: ")
valid_length = len(password) >= 8
has_number = False
for ch in password:
    if ch.isdigit():
        has_number = True
if valid_length and has_number:
    print("Valid")
else:
    print("Invalid")
    if not valid_length:
        print("Too short")
    if not has_number:
        print("Needs a number")

```

Alternative: Using any() function:

```

has_number = any(ch.isdigit() for ch in password)

```

C13. Student Marks Processor (10 marks: 7 pseudocode + 3 Python)

PSEUDODE — 7 marks:

[1 mark] Correct array/list initialisation with all 8 values

[1 mark] Correct loop to calculate sum

[1 mark] Correct average calculation (sum / count)

[1 mark] Correct loop to count passes (mark >= 60)

[1 mark] Find maximum value (loop or built-in)

[1 mark] Correct OUTPUT statements for all required outputs

[1 mark] Pseudocode syntax is correct throughout

Exemplar pseudocode:

Marks ← [65, 72, 58, 91, 84, 76, 55, 88] [1]

Sum ← 0

FOR i ← 1 TO 8 [1]

Sum ← Sum + Marks[i]

NEXT i

Average ← Sum / 8 [1]

PassCount ← 0

FOR i ← 1 TO 8 [1]

IF Marks[i] >= 60 THEN

PassCount ← PassCount + 1

ENDIF

NEXT i

Highest ← Marks[1]

FOR i ← 2 TO 8 [1]

IF Marks[i] > Highest THEN

Highest ← Marks[i]

ENDIF

NEXT i

OUTPUT "Marks: ", Marks [1]

OUTPUT "Average: ", Average

OUTPUT "Passed: ", PassCount

OUTPUT "Highest: ", Highest [1]

PYTHON — 3 marks:

[1 mark] List initialisation

[1 mark] Average and pass count calculations

[1 mark] All outputs correct

Exemplar Python:

marks = [65, 72, 58, 91, 84, 76, 55, 88] [1]

average = sum(marks) / len(marks) [1]

```

pass_count = 0
for mark in marks:
    if mark >= 60:
        pass_count += 1
print("Marks:", marks)
print("Average:", round(average, 1))
print("Passed:", pass_count)
print("Highest:", max(marks))          [1]

```

Alternative for max: use a loop instead of max() — award mark.

SECTION C TOTAL: 20 marks

MOCK EXAM TOTAL: 50 marks

Grade Boundaries (suggested for Y9):

A* (Exceptional): 45–50 marks (90–100%)

A (Secure): 40–44 marks (80–89%)

B (Developing): 32–39 marks (64–79%)

C (Emerging): 25–31 marks (50–63%)

D (Foundation): 20–24 marks (40–49%)

E (Below target): Below 20 marks (<40%)

Note: These are Y9 internal assessment boundaries and are more generous than actual IGCSE grade boundaries. Purpose is to encourage and identify gaps.

STUDENT REPORT TEMPLATE

YEAR 9 COMPUTER SCIENCE — SEMESTER 2 REPORT

Student Name: _____ Class: _____ Term: _____

1. ATTAINMENT SUMMARY

ASSESSMENT	RAW MARK	PERCENTAGE	GRADE / LEVEL
Quiz 1 — Algorithms	___ / 20	→ ___ %	
(Week 3)			
Quiz 2 — Python Programming	___ / 25	→ ___ %	
(Week 7)			
Quiz 3 — Data Structures	___ / 20	→ ___ %	
(Week 10)			
Quiz 4 — File & Functions	___ / 20	→ ___ %	
(Week 14)			
Mock Exam — Paper 2 Style	___ / 50	→ ___ %	

(Week 18)

Classwork & Participation → ____ / 20 → ____ %

(Teacher assessed)

OVERALL SEMESTER GRADE → ____ %

2. TOPIC BREAKDOWN

Rate each topic: E = Emerging | D = Developing | S = Secure

No.	TOPIC	E	D	S
UNIT 1 — ALGORITHMS & PSEUDOCODE				
1	Define and explain algorithms	→ ☐	→ ☐	→ ☐
2	Write pseudocode using CIE conventions	→ ☐	→ ☐	→ ☐
3	Draw and interpret flowcharts	→ ☐	→ ☐	→ ☐
4	Use decomposition to break down problems	→ ☐	→ ☐	→ ☐
5	Apply abstraction to simplify problems	→ ☐	→ ☐	→ ☐
6	Use trace tables to dry-run algorithms	→ ☐	→ ☐	→ ☐
UNIT 2 — PYTHON PROGRAMMING				
7	Declare and use variables correctly	→ ☐	→ ☐	→ ☐
8	Identify and convert between data types	→ ☐	→ ☐	→ ☐
9	Use arithmetic, comparison & logical ops	→ ☐	→ ☐	→ ☐
10	Write IF/ELIF/ELSE selection structures	→ ☐	→ ☐	→ ☐
11	Write FOR loops (count-controlled)	→ ☐	→ ☐	→ ☐
12	Write WHILE loops (condition-controlled)	→ ☐	→ ☐	→ ☐
13	Use nested IF statements and nested loops	→ ☐	→ ☐	→ ☐
UNIT 3 — DATA STRUCTURES				
14	Declare and initialise 1D arrays / lists	→ ☐	→ ☐	→ ☐
15	Access and modify elements by index	→ ☐	→ ☐	→ ☐
16	Perform list operations (append, remove, etc)	→ ☐	→ ☐	→ ☐
17	Use string methods (len, upper, slice, etc)	→ ☐	→ ☐	→ ☐
18	Use ASCII codes (ord, chr)	→ ☐	→ ☐	→ ☐
UNIT 4 — FUNCTIONS				
19	Define and call user-defined functions	→ ☐	→ ☐	→ ☐
20	Distinguish functions from procedures	→ ☐	→ ☐	→ ☐
21	Use parameters and arguments correctly	→ ☐	→ ☐	→ ☐
22	Understand and apply scope (local/global)	→ ☐	→ ☐	→ ☐
UNIT 5 — FILE HANDLING				
23	Open and close files correctly	→ ☐	→ ☐	→ ☐
24	Read data from text files	→ ☐	→ ☐	→ ☐

25 → Write data to text files → ☐ → ☐ → ☐

26 → Understand CSV file structure → ☐ → ☐ → ☐

UNIT 6 — TESTING & DOCUMENTATION

27 → Identify syntax, logic and runtime errors → ☐ → ☐ → ☐

28 → Design test plans with appropriate test data → ☐ → ☐ → ☐

29 → Create trace tables for code verification → ☐ → ☐ → ☐

30 → Add comments and documentation to code → ☐ → ☐ → ☐

Topic Rating Summary:

Secure topics: ____ / 30

Developing topics: ____ / 30

Emerging topics: ____ / 30

3. STRENGTHS

- ☐ Algorithm design and pseudocode writing
- ☐ Python programming (selection and iteration)
- ☐ Data structure manipulation (lists and strings)
- ☐ Function definition and use
- ☐ File handling operations
- ☐ Testing and error identification
- ☐ Code tracing and dry running
- ☐ Problem decomposition

Specific comments:

4. AREAS FOR DEVELOPMENT

5. TARGETS FOR YEAR 10

Target 1: _____

Target 2: _____

Target 3: _____

6. TEACHER COMMENT

Teacher signature: _____ Date: _____

7. PARENT/CARER ACKNOWLEDGEMENT

I have seen this report.

Parent signature: _____ Date: _____

END OF REPORT TEMPLATE

BILINGUAL GLOSSARY POSTER

50+ PROGRAMMING TERMS WITH CHINESE & PINYIN

This glossary is designed to be printed as A3 wall posters (one per unit)

or as A4 handouts for student folders. Colour-code by unit:

Unit 1 (Algorithms) — BLUE

Unit 2 (Python Basics) — GREEN

Unit 3 (Data Structures)— ORANGE

Unit 4 (Functions) — PURPLE

Unit 5 (File Handling) — RED

Unit 6 (Testing) — GREY

UNIT 1 — ALGORITHMS & PROBLEM SOLVING | BLUE

ENGLISH	→	中文 (Chinese)	→	PINYIN
algorithm	→	算法	→	suàn fǎ
pseudocode	→	伪代码	→	wěi dài mǎ
flowchart	→	流程图	→	liú chéng tú
decomposition	→	分解	→	fēn jiě
abstraction	→	抽象	→	chōu xiàng
stepwise refinement	→	逐步细化	→	zhú bù xì huà
dry run	→	手工运行	→	shǒu gōng yùn xíng
trace table	→	跟踪表	→	gēn zōng biǎo
terminal symbol	→	起止符	→	qǐ zhǐ fú
process symbol	→	处理符	→	chǔ lǐ fú
decision symbol	→	判断符	→	pàn duàn fú
input/output symbol	→	输入/输出符	→	shū rù / shū chū fú
flow line	→	流程线	→	liú chéng xiàn
subroutine	→	子程序	→	zǐ chéng xù
algorithmic thinking	→	算法思维	→	suàn fǎ sī wéi

UNIT 2 — PYTHON BASICS & CONTROL STRUCTURES | GREEN

	→		→	
--	---	--	---	--

ENGLISH → 中文 (Chinese) → PINYIN

variable → 变量 → biàn liàng

constant → 常量 → cháng liàng

data type → 数据类型 → shù jù lèi xíng

integer → 整数 → zhěng shù

real / float → 实数 / 浮点数 → shí shù / fú diǎn shù

string → 字符串 → zì fú chuàn

Boolean → 布尔值 → bù ěr zhí

character (char) → 字符 → zì fú

input → 输入 → shū rù

output → 输出 → shū chū

assignment → 赋值 → fù zhí

comment → 注释 → zhù shì

arithmetic operator → 算术运算符 → suàn shù yùn suàn fú

comparison operator → 比较运算符 → bǐ jiào yùn suàn fú

logical operator → 逻辑运算符 → luó jí yùn suàn fú

AND → 与 / 并且 → yǔ / bìng qiě

OR → 或 / 或者 → huò / huò zhě

NOT → 非 / 不 → fēi / bù

MOD → 取模 / 求余 → qǔ mó / qiú yú

DIV → 整除 → zhěng chú

selection → 选择结构 → xuǎn zé jié gòu

if → 如果 → rú guǒ

else → 否则 → fǒu zé

elif / else if → 否则如果 → fǒu zé rú guǒ

condition → 条件 → tiáo jiàn

nested → 嵌套的 → qiàn tào de

iteration → 迭代 / 循环 → dié dài / xún huán

for loop → for 循环 → for xún huán

while loop → while 循环 → while xún huán

range → 范围 → fàn wéi

infinite loop → 无限循环 → wú xiàn xún huán

nested loop → 嵌套循环 → qiàn tào xún huán

indentation → 缩进 → suō jìn

code block → 代码块 → dài mǎ kuài

type conversion / casting → 类型转换 → lèi xíng zhuǎn huàn
counter-controlled loop → 计数控制循环 → jì shù kòng zhì xún huán
condition-controlled loop → 条件控制循环 → tiáo jiàn kòng zhì xún huán
post-condition loop → 后测试循环 → hòu cè shì xún huán
pre-condition loop → 前测试循环 → qián cè shì xún huán

UNIT 3 — DATA STRUCTURES: ARRAYS & STRINGS | ORANGE

ENGLISH → 中文 (Chinese) → PINYIN
array → 数组 → shù zǔ
list (Python) → 列表 → liè biǎo
index → 索引 / 下标 → suǒ yǐn / xià biāo
element → 元素 → yuán sù
2D array → 二维数组 → èr wéi shù zǔ
append → 添加 / 追加 → tiān jiā / zhuī jiā
remove → 删除 → shān chú
insert → 插入 → chā rù
length → 长度 → cháng dù
concatenation → 连接 / 拼接 → lián jiē / pīn jiē
slicing → 切片 → qiē piàn
substring → 子字符串 → zǐ zì fú chuàn
ASCII → ASCII 码 → ASCII mǎ
ORD() → 取 ASCII 码 → qǔ ASCII mǎ
CHR() → 取字符 → qǔ zì fú
searching → 搜索 → sōu suǒ
linear search → 线性搜索 → xiàn xìng sōu suǒ
traversing → 遍历 → biàn lì

UNIT 4 — FUNCTIONS & PROCEDURES | PURPLE

ENGLISH → 中文 (Chinese) → PINYIN
function → 函数 → hán shù

procedure → 过程 / 子程序 → guò chéng / zǐ chéng xù
parameter → 形参 / 参数 → xíng cān / cān shù
argument → 实参 / 参数值 → shí cān / cān shù zhí
return value → 返回值 → fǎn huí zhí
scope → 作用域 → zuò yòng yù
local variable → 局部变量 → jú bù biàn liàng
global variable → 全局变量 → quán jú biàn liàng
module → 模块 → mó kuài
library / library routine → 库 / 函数库 → kù / hán shù kù
import → 导入 → dǎo rù
built-in function → 内置函数 → nèi zhì hán shù
user-defined function → 用户定义函数 → yòng hù dìng yì hán shù
call / invoke → 调用 → diào yòng
pass by value → 值传递 → zhí chuán dì

UNIT 5 — FILE HANDLING | RED

ENGLISH → 中文 (Chinese) → PINYIN
file → 文件 → wén jiàn
read → 读取 → dú qǔ
write → 写入 → xiě rù
append → 追加 → zhuī jiā
open → 打开 → dǎ kāi
close → 关闭 → guān bì
CSV → CSV 格式 → CSV gé shì
file mode → 文件模式 → wén jiàn mó shì
EOF (End Of File) → 文件结束 → wén jiàn jié shù
record → 记录 → jì lù
field → 字段 → zì duàn
delimiter → 分隔符 → fēn gé fú

UNIT 6 — TESTING, DEBUGGING & DOCUMENTATION | GREY

ENGLISH → 中文 (Chinese) → PINYIN

test plan → 测试计划 → cè shì jì huà

dry run → 手工运行 → shǒu gōng yùn xíng

syntax error → 语法错误 → yǔ fǎ cuò wù

logic error → 逻辑错误 → luó jí cuò wù

runtime error → 运行时错误 → yùn xíng shí cuò wù

normal data → 正常数据 → zhèng cháng shù jù

boundary data → 边界数据 → biān jiè shù jù

erroneous data → 错误数据 → cuò wù shù jù

invalid data → 无效数据 → wú xiào shù jù

validation → 验证 → yàn zhèng

verification → 确认 → què rèn

bug → 程序错误 → chéng xù cuò wù

debugging → 调试 → tiáo shì

analysis → 分析 → fēn xī

design → 设计 → shè jì

coding → 编码 → biān mǎ

testing → 测试 → cè shì

maintenance → 维护 → wéi hù

evaluation → 评估 → píng gū

documentation → 文档 → wén dǎng

internal documentation → 内部文档 → nèi bù wén dǎng

external documentation → 外部文档 → wài bù wén dǎng

systems life cycle → 系统生命周期 → xì tǒng shēng mìng zhōu qí

GLOSSARY STATISTICS

Category → Count

Unit 1 — Algorithms & Problem Solving → 15

Unit 2 — Python Basics & Control → 37

Unit 3 — Data Structures → 18

Unit 4 — Functions & Procedures → 15

Unit 5 — File Handling → 12

Unit 6 — Testing, Debugging & Docs → 23

TOTAL UNIQUE TERMS → 88+

Note: Some terms appear across multiple units (e.g., "append" in both lists and file handling). The glossary includes all context-specific uses.

QUICK REFERENCE — PINYIN PRONUNCIATION GUIDE

For teachers who do not read Chinese, here is a quick pronunciation guide:

- sh = like "sh" in "ship" (tongue further back)
- zh = like "j" in "judge" (tongue curled back)
- ch = like "ch" in "church" (tongue curled back)
- c = like "ts" in "cats"
- q = like "ch" in "cheese" (tongue tip down)
- x = like "sh" in "she" (tongue tip down)
- z = like "dz" in "adze"
- r = like "r" in "run" (tongue curled back slightly)
- ü = round lips as for "oo" but say "ee"

TONES (shown by macrons in pinyin):

- 1st tone: high and level (mā) — ā, ē, ī, ō, ū, ŭ
- 2nd tone: rising (má) — á, é, í, ó, ú, ŭ
- 3rd tone: falling then rising (mǎ) — ǎ, ě, ĭ, ǒ, ǔ, ǔ
- 4th tone: falling (mà) — à, è, ì, ò, ù, ù

END OF BILINGUAL GLOSSARY POSTER

WORD BANK FOR ASSESSMENTS (BILINGUAL)

To be printed on the reverse of each quiz paper

QUIZ 1 WORD BANK — Algorithms & Pseudocode

- algorithm 算法 (suàn fǎ) | pseudocode 伪代码 (wěi dài mǎ)
- flowchart 流程图 (liú chéng tú) | decomposition 分解 (fēn jiě)
- abstraction 抽象 (chōu xiàng) | trace table 跟踪表 (gēn zōng biǎo)
- dry run 手工运行 (shǒu gōng yùn xíng) | terminal 起止符 (qǐ zhǐ fú)
- process 处理符 (chǔ lǐ fú) | decision 判断符 (pàn duàn fú)
- input/output 输入/输出 (shū rù/shū chū) | flow line 流程线 (liú chéng xiàn)
- WHILE loop WHILE 循环 | FOR loop FOR 循环
- REPEAT UNTIL 重复直到 | IF ... THEN ... ELSE 如果...那么...否则
- assignment 赋值 (←) | OUTPUT 输出

QUIZ 2 WORD BANK — Python Programming

variable 变量 (biàn liàng) | data type 数据类型 (shù jù lèi xíng)
integer 整数 (zhěng shù) | float 浮点数 (fú diǎn shù)
string 字符串 (zì fú chuàn) | Boolean 布尔值 (bù ěr zhí)
input 输入 (shū rù) | output 输出 (shū chū)
if / else / elif 如果/否则/否则如果 | for loop for 循环 (for xún huán)
while loop while 循环 (while xún huán) | range 范围 (fàn wéi)
comment 注释 (zhù shì) | syntax error 语法错误 (yǔ fǎ cuò wù)
indentation 缩进 (suō jìn) | MOD 取模 (qǔ mó)

QUIZ 3 WORD BANK — Data Structures

array 数组 (shù zǔ) | list 列表 (liè biǎo)
index 索引 (suǒ yǐn) | element 元素 (yuán sù)
append 添加 (tiān jiā) | remove 删除 (shān chú)
length 长度 (cháng dù) | concatenation 连接 (lián jiē)
slicing 切片 (qiē piàn) | substring 子字符串 (zǐ zì fú chuàn)
ASCII ASCII 码 | 2D array 二维数组 (èr wéi shù zǔ)
search 搜索 (sōu suǒ) | linear search 线性搜索 (xiàn xìng sōu suǒ)

QUIZ 4 WORD BANK — File Handling & Functions

function 函数 (hán shù) | parameter 参数 (cān shù)
argument 参数值 (cān shù zhí) | return 返回 (fǎn huí)
scope 作用域 (zuò yòng yù) | local variable 局部变量 (jú bù biàn liàng)
global variable 全局变量 (quán jú biàn liàng) | module 模块 (mó kuài)
import 导入 (dǎo rù) | file 文件 (wén jiàn)
open 打开 (dǎ kāi) | close 关闭 (guān bì)
read 读取 (dú qǔ) | write 写入 (xiě rù)
append mode 追加模式 (zhuī jiā mó shì) | CSV CSV 格式
file mode 文件模式 (wén jiàn mó shì) | EOF 文件结束 (wén jiàn jié shù)

MOCK EXAM WORD BANK — Mixed Topics

algorithm 算法 | pseudocode 伪代码
flowchart 流程图 | decomposition 分解
variable 变量 | constant 常量
data type 数据类型 | integer 整数
string 字符串 | Boolean 布尔值
selection 选择结构 | iteration 循环

for loop for 循环	while loop while 循环
array 数组	list 列表
index 索引	function 函数
parameter 参数	return value 返回值
scope 作用域	local 局部的
global 全局的	file 文件
read 读取	write 写入
syntax error 语法错误	logic error 逻辑错误
test plan 测试计划	trace table 跟踪表
normal data 正常数据	boundary data 边界数据
erroneous data 错误数据	validation 验证
comment 注释	documentation 文档

END OF ASSESSMENT PACK

Document: VPA-CS-Y9S2-ASSESS-v1.0

Prepared: Victoria Park Academy Shenzhen — Computer Science Department

Syllabus: CIE IGCSE Computer Science (0478)

For Use: Academic Year 2025–2026, Year 9 Semester 2

Contents: 4 Quizzes (85 marks total) + 1 Mock Exam (50 marks) +
Report Template + Bilingual Glossary (88 terms)